

瞭解 Interaction to Next Paint (INP)

來源：<https://codelabs.developers.google.com/understanding-inp?hl=zh-tw>

步驟數：18

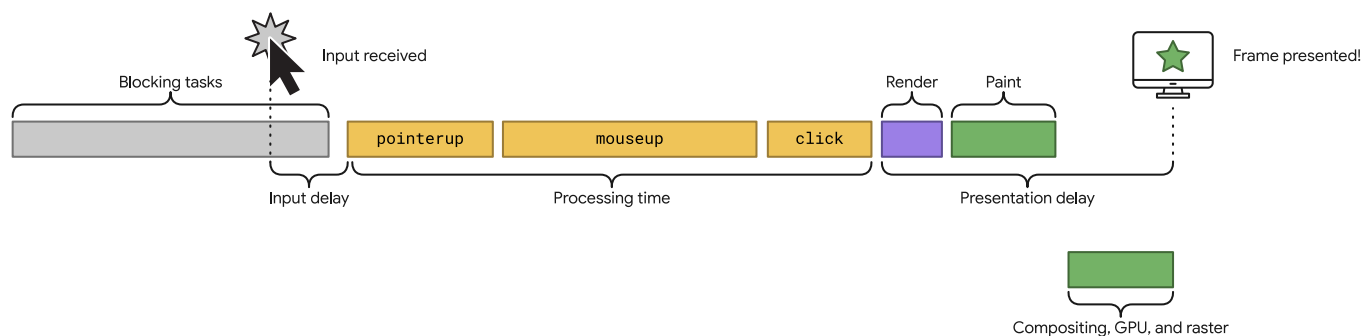
目錄

1. 簡介
2. 做好準備
3. 使用 Chrome 開發人員工具評估互動
4. 長時間執行的事件監聽器
5. 實驗：處理時間
6. 處理時間實驗結果
7. Experiment: input delay
8. 輸入延遲實驗結果
9. 簡報速度緩慢
10. 實驗：簡報延遲
11. 簡報延遲實驗結果
12. 診斷互動
13. 實驗：非同步工作
14. 非同步工作實驗結果
15. 多次互動 (和暴怒點擊)
16. 策略 1：去抖動
17. 策略 2：中斷長時間執行工作
18. 結論

步驟 1 · 約 0 分鐘

1. 簡介

本程式碼研究室會以互動示範的方式，介紹 [Interaction to Next Paint \(INP\)](#)。



必要條件

- 熟悉 HTML 和 JavaScript 開發。
- 建議：參閱 [INP 說明文件](#)。

課程內容

- 使用者互動和您處理這些互動的方式，如何影響網頁回應速度。
- 如何減少及消除延遲，提供流暢的使用體驗。

需求條件

- 電腦必須能夠從 GitHub 複製程式碼，並執行 npm 指令。
 - 文字編輯器。
 - 使用新版 Chrome，才能正常進行所有互動評估。
-

步驟 2 · 約 0 分鐘

2. 做好準備

取得並執行程式碼

程式碼位於 [web-vitals-codelabs](#) 存放區。

1. 在終端機中複製存放區：`git clone https://github.com/GoogleChromeLabs/web-vitals-codelabs.git`
2. 前往複製的目錄：`cd web-vitals-codelabs/understanding-inp`
3. 安裝依附元件：`npm ci`
4. 啟動網路伺服器：`npm run start`
5. 在瀏覽器中前往 <http://localhost:5173/understanding-inp/>

應用程式總覽

頁面頂端會顯示「分數」計數器和「增加」按鈕。這是反應性和回應性的經典示範！

Score: 12

Increment

INP

56

Interaction

40

FPS

60

Timer

19.79

按鈕下方有四項測量結果：

- INP：目前的 INP 分數，通常是最差的互動。
- 互動：最近一次互動的分數。
- FPS：網頁主執行緒的每秒影格數。
- 計時器：顯示執行中的計時器動畫，協助您瞭解卡頓情形。

測量互動時，完全不需要 FPS 和計時器項目。新增這些項目只是為了方便您瞭解回應式設計。

立即試用

試著與「Increment」按鈕互動，並觀察分數是否增加。**INP** 和 **互動**值是否會隨著每次增量而改變？

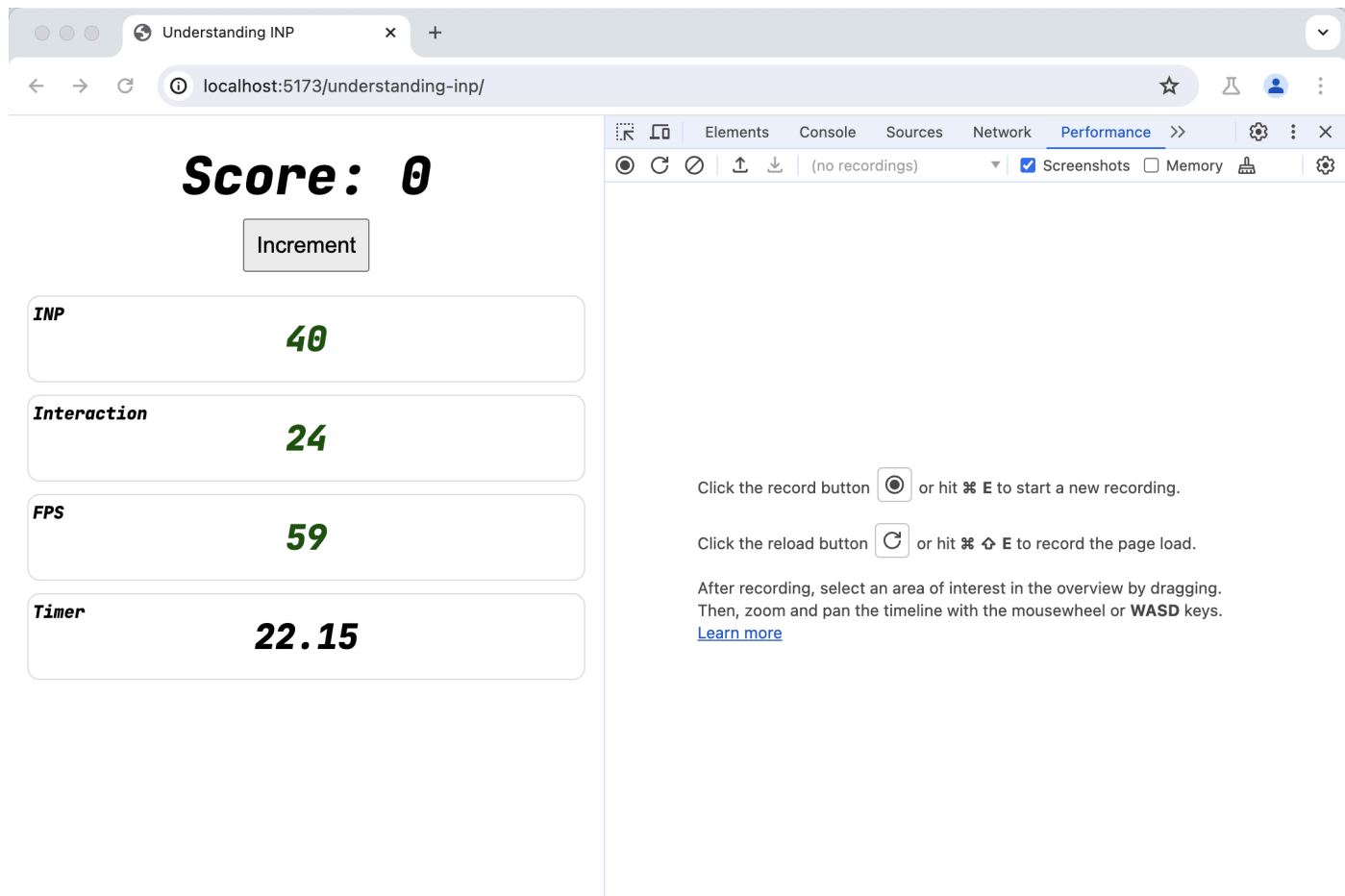
INP 會測量從使用者互動到網頁實際向使用者顯示更新內容的時間。

步驟 3 · 約 0 分鐘

3. 使用 Chrome 開發人員工具評估互動

開啟開發人員工具：依序選取「更多工具」 > 「開發人員工具」選單；在網頁上按一下滑鼠右鍵，然後選取「檢查」；或使用鍵盤快速鍵。

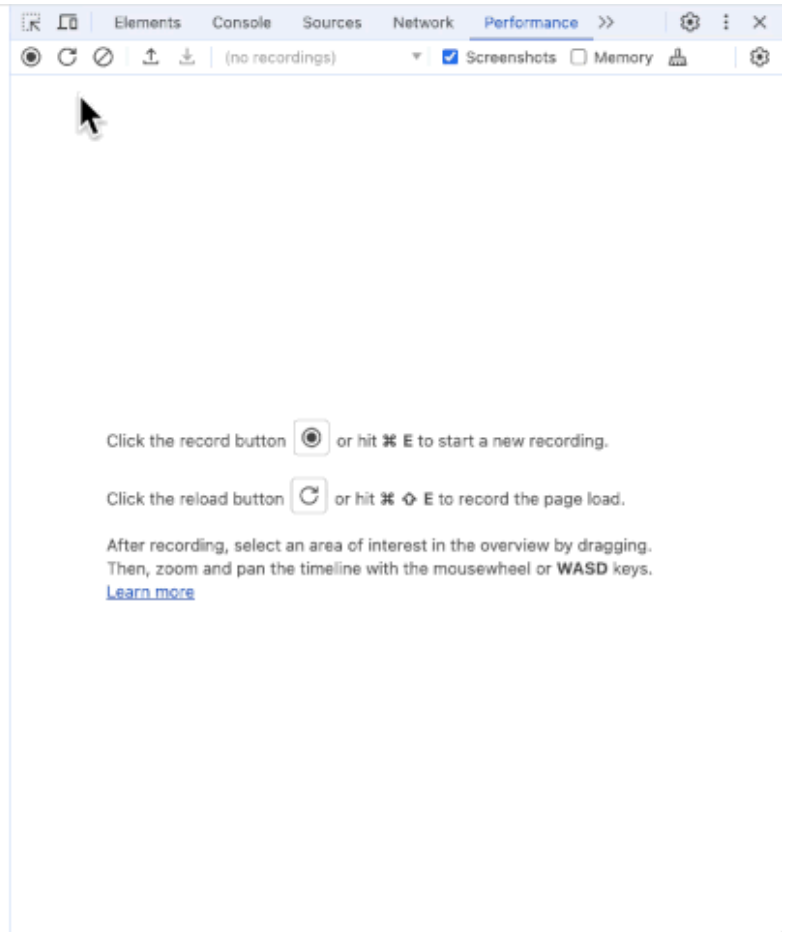
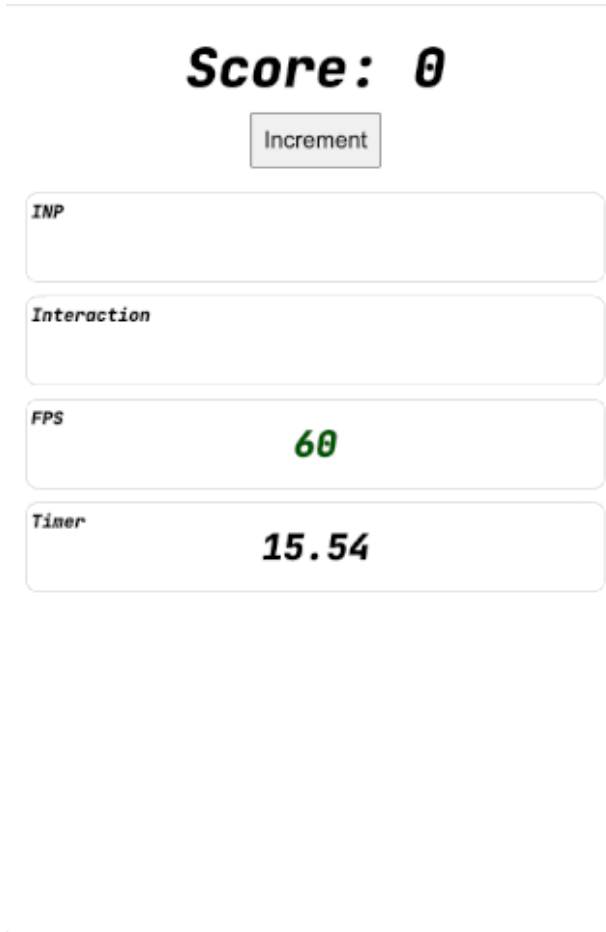
切換至「效能」面板，用來評估互動。



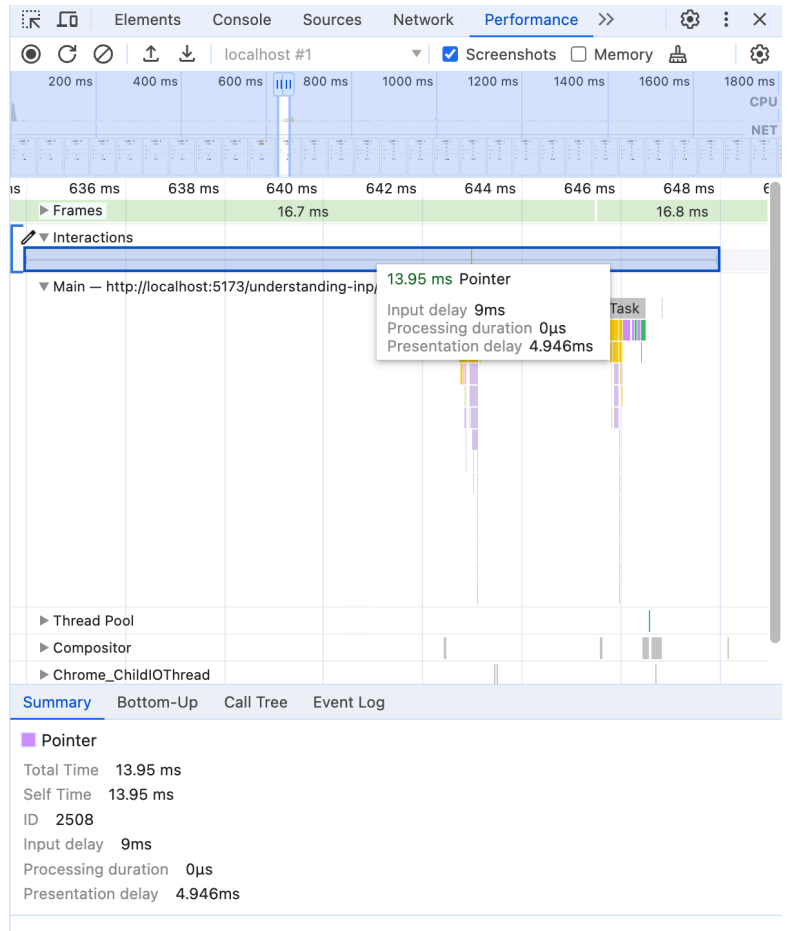
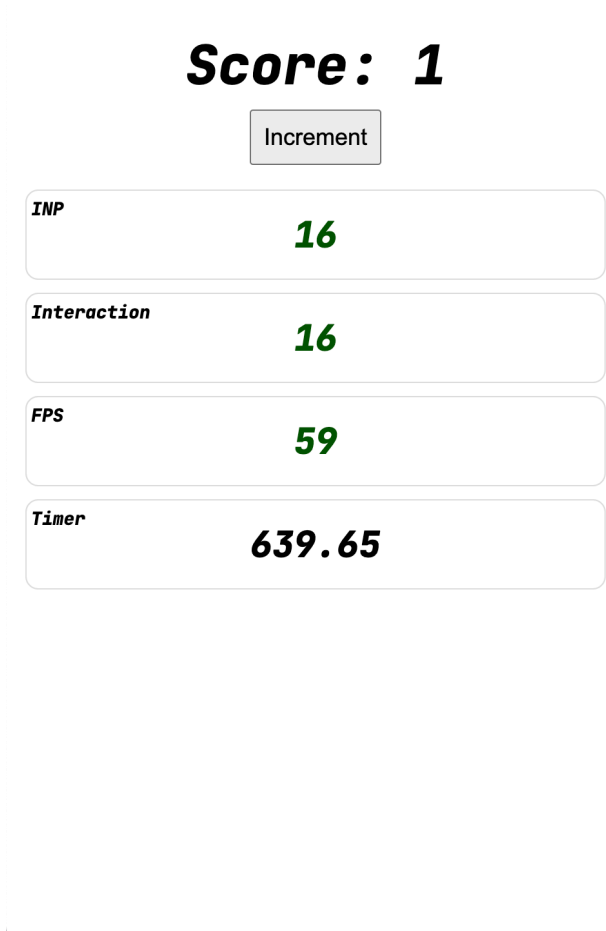
接著，在「效能」面板中擷取互動。

1. 按下 [錄製]。
2. 與網頁互動 (按下「Increment」按鈕)。
3. 停止錄製。

時間軸隨即會顯示「互動」軌。按一下左側的三角形即可展開。



畫面會顯示兩項互動。捲動或按住 W 鍵，即可放大第二個畫面。



將滑鼠游標懸停在互動上，您會看到互動速度很快，完全沒有處理時間，且輸入延遲和顯示延遲時間最短，實際長度取決於電腦速度。

步驟 4 · 約 0 分鐘

4. 長時間執行的事件監聽器

開啟 `index.js` 檔案，並取消事件監聽器中 `blockFor` 函式的註解。

[查看完整程式碼：click_block.html](#)

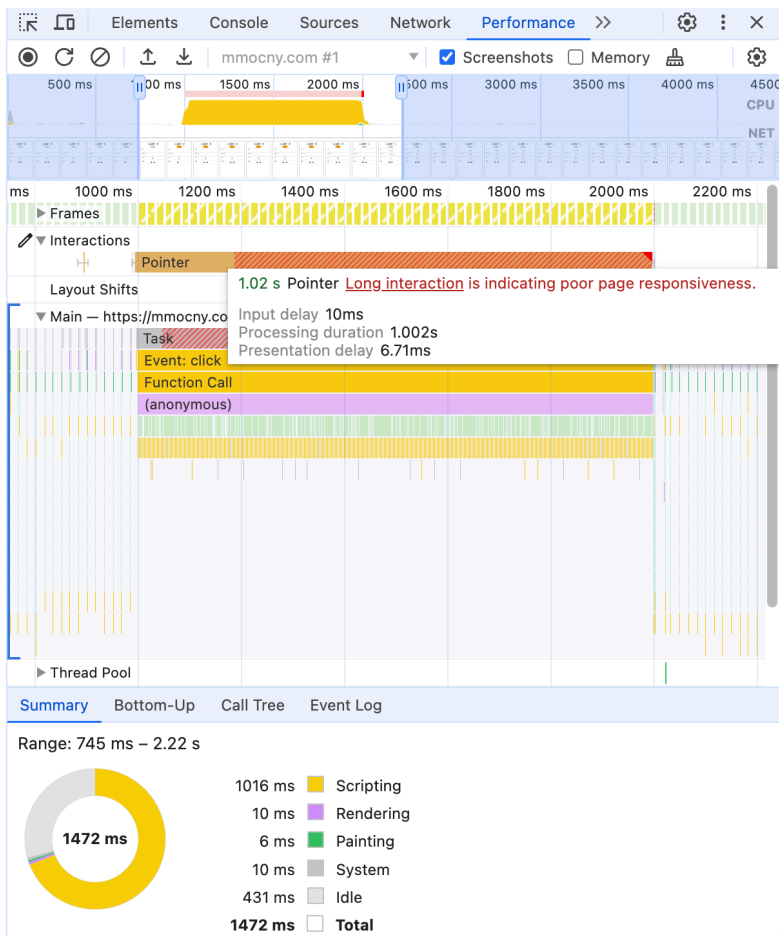
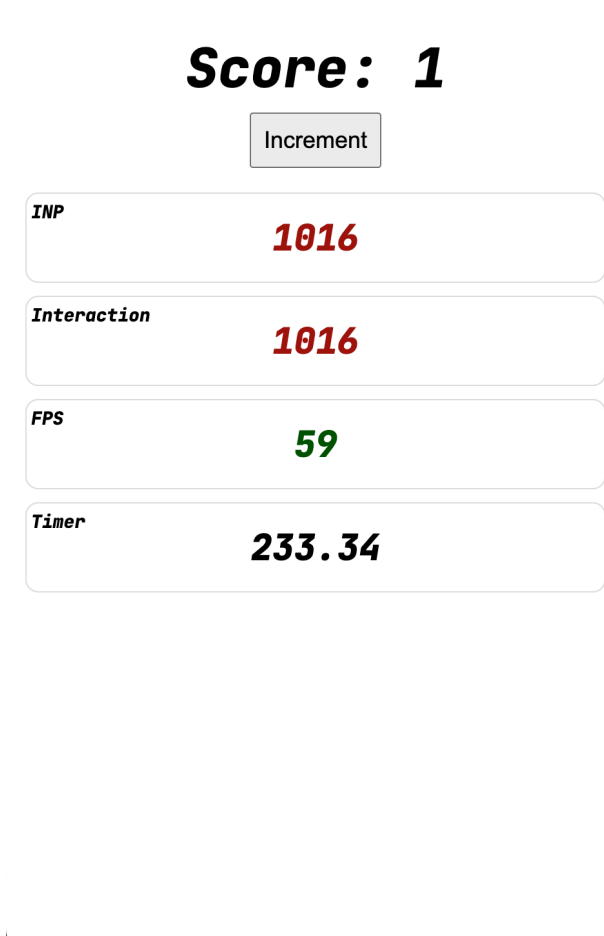
```
button.addEventListener('click', () => {
  blockFor(1000);
  score.incrementAndUpdateUI();
});
```

儲存檔案。伺服器會看到這項變更，並為您重新整理頁面。

請再次與頁面互動。現在互動速度會明顯變慢。

效能追蹤

在「效能」面板中再次錄製，看看會顯示什麼內容。



原本短暫的互動現在需要一整秒。

將滑鼠游標懸停在互動上，會發現時間幾乎都花在「處理時間」上，這是執行事件監聽器回呼所花費的時間。由於封鎖 `blockFor` 呼叫完全位於事件監聽器內，因此時間會花費在該處。

步驟 5 · 約 0 分鐘

5. 實驗：處理時間

嘗試重新安排事件監聽器工作，看看對 INP 有何影響。

請先更新 UI

如果調換 JavaScript 呼叫的順序，先更新 UI 再封鎖，會發生什麼情況？

[查看完整程式碼：ui_first.html](#)

```
button.addEventListener('click', () => {  
  score.incrementAndUpdateUI();  
  blockFor(1000);  
});
```

您是否注意到先前出現的 UI？順序會影響 INP 分數嗎？

請嘗試擷取追蹤記錄並檢查互動，看看是否有任何差異。

獨立監聽器

如果將工作移至個別事件監聽器，會發生什麼情況？在一個事件監聽器中更新 UI，並在另一個監聽器中封鎖網頁。

[查看完整程式碼：two_click.html](#)

```
button.addEventListener('click', () => {  
  score.incrementAndUpdateUI();  
});  
  
button.addEventListener('click', () => {  
  blockFor(1000);  
});
```

現在效能面板中會顯示什麼？

不同事件類型

大多數互動都會觸發多種事件，包括指標或按鍵事件、懸停、焦點/模糊，以及 beforechange 和 beforeinput 等合成事件。

許多實際網頁都有多個不同事件的監聽器。

如果變更事件監聽器的事件類型，會發生什麼情況？舉例來說，您是否要將其中一個 `click` 事件監聽器替換為 `pointerup` 或 `mouseup`？

[查看完整程式碼：diff_handlers.html](#)

```
button.addEventListener('click', () => {
  score.incrementAndUpdateUI();
});

button.addEventListener('pointerup', () => {
  blockFor(1000);
});
```

不更新 UI

如果從事件監聽器中移除更新 UI 的呼叫，會發生什麼情況？

[查看完整程式碼：no_ui.html](#)

```
button.addEventListener('click', () => {
  blockFor(1000);
  // score.incrementAndUpdateUI();
});
```

步驟 6 · 約 0 分鐘

6. 處理時間實驗結果

效能追蹤：先更新 UI

查看完整程式碼：[ui_first.html](#)

```
button.addEventListener('click', () => {
  score.incrementAndUpdateUI();
  blockFor(1000);
});
```

查看點擊按鈕的「效能」面板記錄，您會發現結果沒有變更。雖然在阻斷程式碼之前觸發了 UI 更新，但瀏覽器直到事件監聽器完成後，才實際更新繪製到螢幕上的內容，這表示互動仍需一秒以上才能完成。

Score: 1

Increment

INP

1024

Interaction

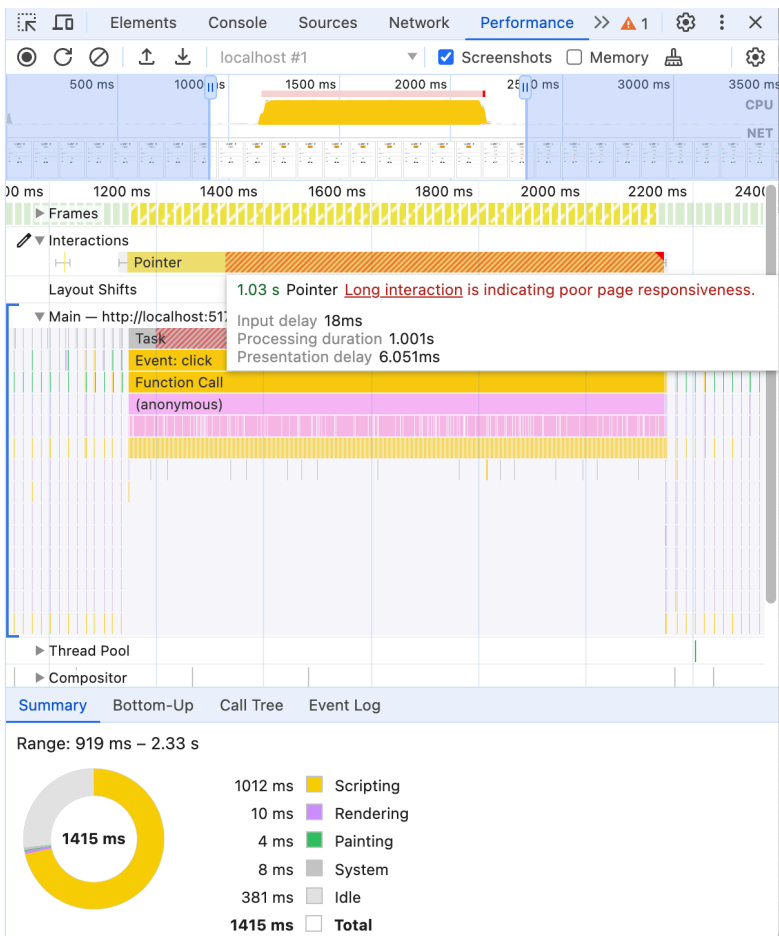
1024

FPS

59

Timer

14.28



效能追蹤記錄：個別監聽器

查看完整程式碼：[two_click.html](#)

```
button.addEventListener('click', () => {
  score.incrementAndUpdateUI();
});

button.addEventListener('click', () => {
  blockFor(1000);
});
```

同樣地，這兩者在功能上並無不同。互動時間仍為一秒。

如果將點擊互動放大，您會發現由於 `click` 事件，確實有兩個不同的函式遭到呼叫。

如預期，第一個 (更新 UI) 的執行速度非常快，第二個則需要整整一秒。但這些效果加總起來，最終仍會導致使用者互動緩慢。

Score: 2

Increment

INP

1032

Interaction

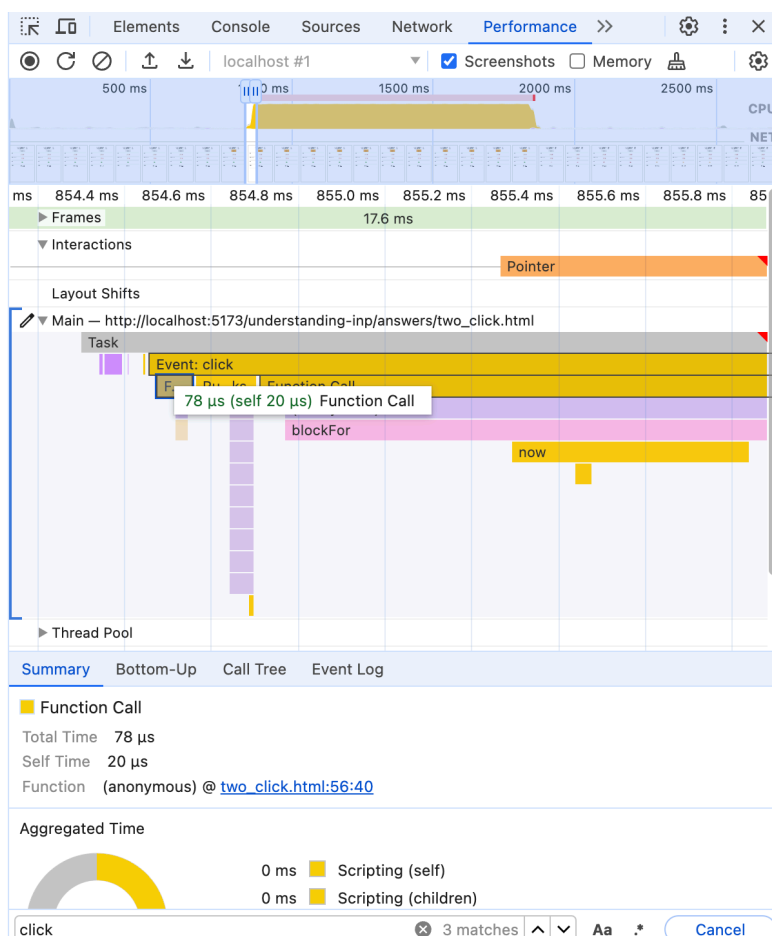
1032

FPS

59

Timer

314.29

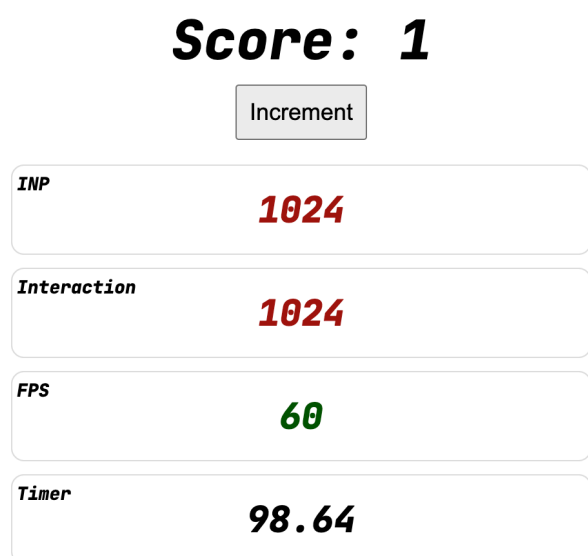


效能追蹤：不同事件類型

```
button.addEventListener('click', () => {
  score.incrementAndUpdateUI();
});

button.addEventListener('pointerup', () => {
  blockFor(1000);
});
```

這些結果非常相似。互動時間仍為一秒，唯一的差別在於較短的 UI 更新專用 `click` 監聽器現在會在封鎖 `pointerup` 監聽器之後執行。



效能追蹤：沒有 UI 更新

[查看完整程式碼：no_ui.html](#)

```
button.addEventListener('click', () => {
  blockFor(1000);
  // score.incrementAndUpdateUI();
});
```

- 分數不會更新，但網頁仍會更新！
- 動畫、CSS 效果、預設網頁元件動作 (表單輸入)、文字輸入、文字醒目顯示都會持續更新。

在這種情況下，按鈕會進入有效狀態，並在點選後返回，這需要瀏覽器繪製，因此仍有 INP。

由於事件監聽器封鎖了主要執行緒一秒，導致網頁無法繪製，因此互動仍需一秒。

錄製「效能」面板時，互動方式與先前幾乎相同。

Score: 0

Increment

INP

1016

Interaction

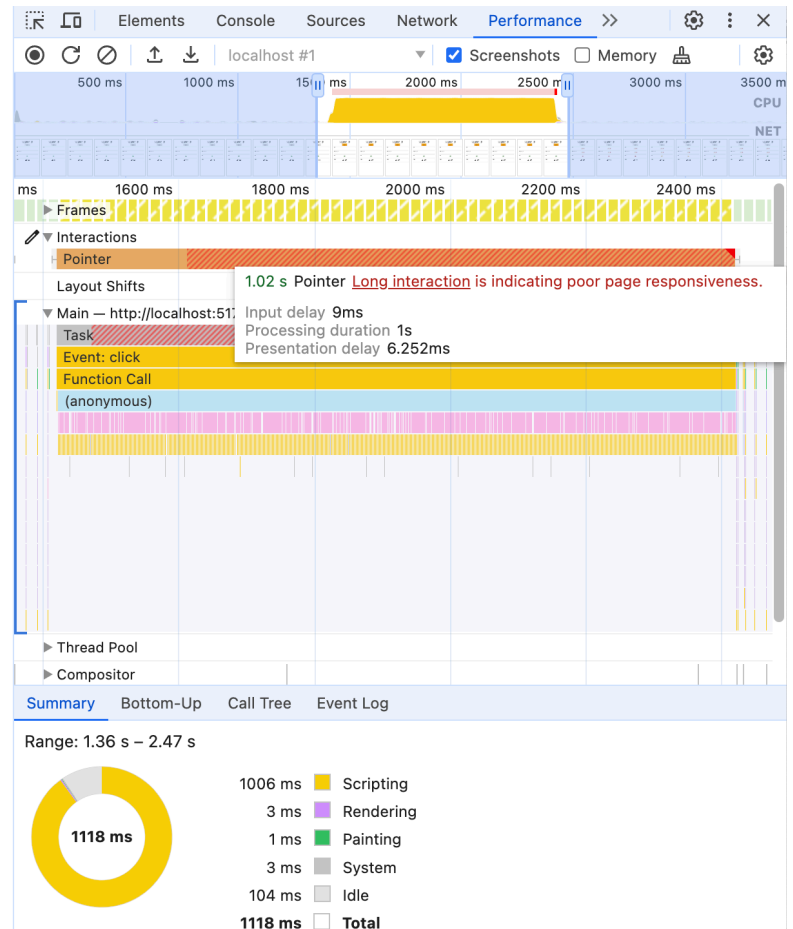
1016

FPS

60

Timer

22.84



外帶

在任何事件監聽器中執行的任何程式碼，都會延遲互動。

- 包括從不同指令碼註冊的事件監聽器，以及在事件監聽器中執行的架構或程式庫程式碼，例如觸發元件算繪的狀態更新。
- 不僅是您自己的程式碼，也包括所有第三方指令碼。

這是常見問題！

最後：程式碼不會觸發繪製，並不代表繪製不會等待緩慢的事件監聽器完成。

7. Experiment: input delay

如果事件監聽器以外的程式碼長時間執行，會發生什麼情況？例如：

- 如果載入時間較晚的 `<script>` 在載入期間隨機封鎖網頁。
- 定期封鎖網頁的 API 呼叫 (例如 `setInterval()`)？

請嘗試從事件監聽器中移除 `blockFor`，並將其新增至 `setInterval()`：

[查看完整程式碼：input_delay.html](#)

```
setInterval(() => {  
  blockFor(1000);  
}, 3000);  
  
button.addEventListener('click', () => {  
  score.incrementAndUpdateUI();  
});
```

說明/活動

步驟 8 · 約 0 分鐘

8. 輸入延遲實驗結果

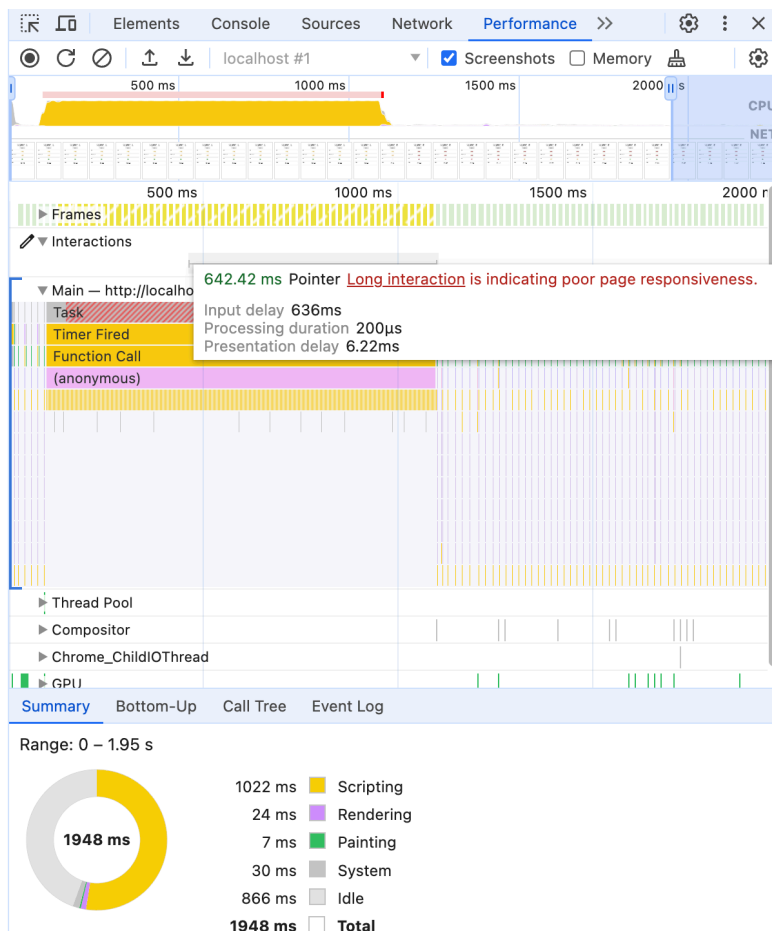
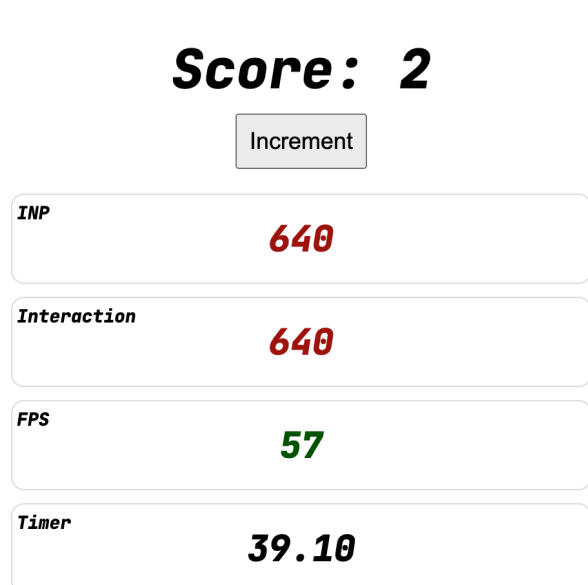
查看完整程式碼：input_delay.html

```
setInterval(() => {  
  blockFor(1000);  
}, 3000);  
  
button.addEventListener('click', () => {  
  score.incrementAndUpdateUI();  
});
```

如果在 `setInterval` 封鎖工作執行期間記錄按鈕點擊，即使互動本身沒有封鎖工作，也會導致長時間互動！

這類長時間執行的作業通常稱為「長時間工作」。

將滑鼠游標懸停在開發人員工具中的互動上，您會發現互動時間現在主要歸因於輸入延遲，而非處理時間。



請注意，這不一定會影響互動！如果工作執行時沒有點選，或許會幸運地獲得獎勵。如果這類「隨機」打噴嚏只會偶爾造成問題，偵錯時可能會非常棘手。

如要找出這些問題，可以測量長時間任務 (或長動畫影格) 和 **Total Blocking Time**。

9. 簡報速度緩慢

到目前為止，我們已透過輸入延遲或事件監聽器查看 JavaScript 的效能，但還有哪些因素會影響下一次算繪？

沒錯，就是用昂貴的特效更新頁面！

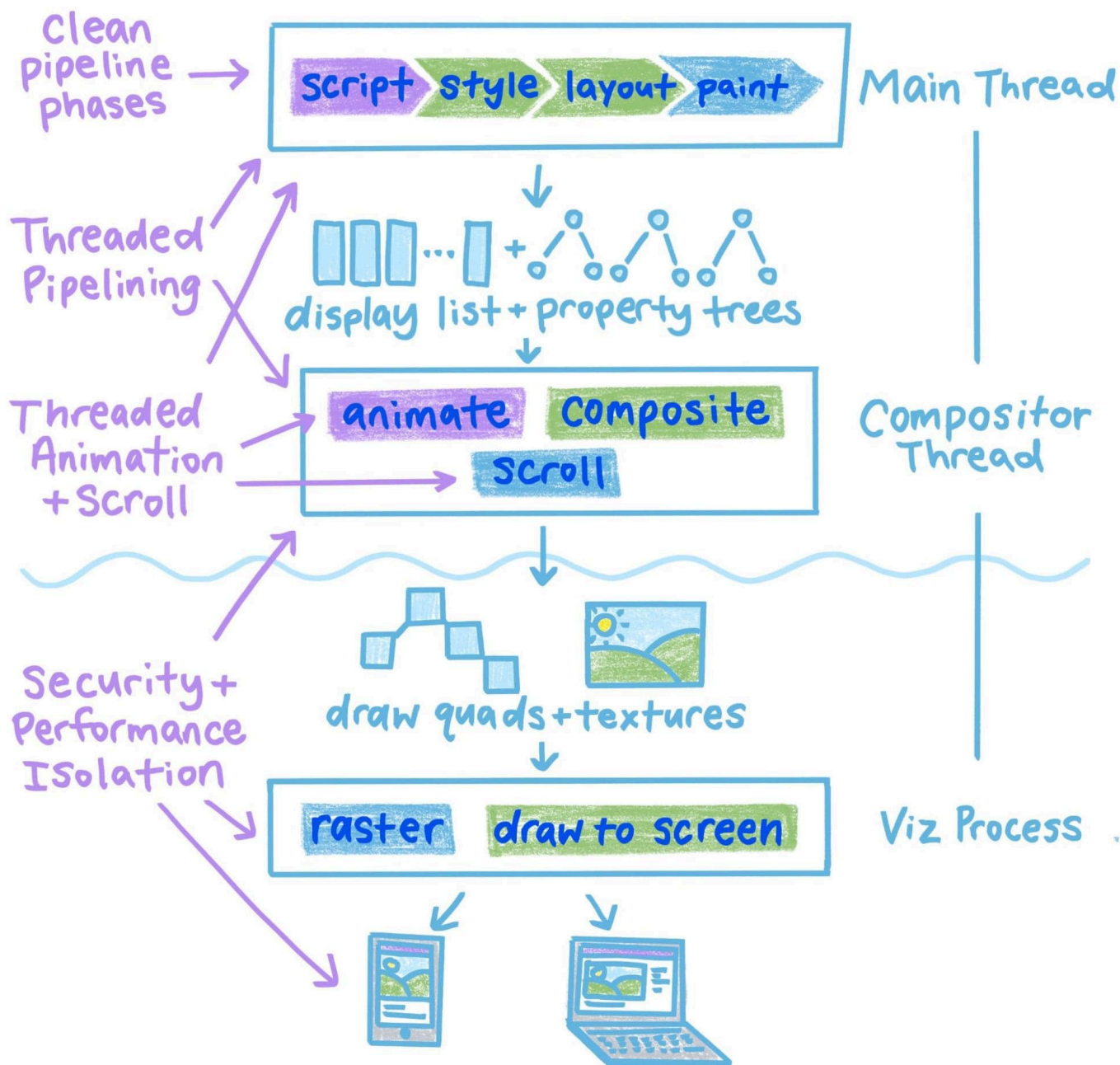
即使網頁更新速度很快，瀏覽器可能仍需費力轉譯！

在主執行緒上：

- 需要在狀態變更後算繪更新內容的 UI 架構
- DOM 變更或切換許多耗用資源的 CSS 查詢選擇器，可能會觸發大量樣式、版面配置和繪製作業。

在主執行緒外：

- 使用 CSS 驅動 GPU 效果
- 加入非常大的高解析度圖片
- 使用 SVG/Canvas 繪製複雜場景



RenderingNG

以下列舉一些常見的網路範例：

- SPA 網站會在點選連結後重建整個 DOM，不會暫停提供初始視覺回饋。
- 搜尋頁面提供複雜的搜尋篩選器和動態使用者介面，但執行這些功能需要耗費大量監聽器。
- 深色模式切換按鈕，可觸發整個頁面的樣式/版面配置

步驟 10 · 約 0 分鐘

10. 實驗：簡報延遲

`requestAnimationFrame` 速度緩慢

讓我們使用 `requestAnimationFrame()` API 模擬簡報延遲時間較長的情況。

將 `blockFor` 呼叫移至 `requestAnimationFrame` 回呼中，以便在事件監聽器傳回後執行：

[查看完整程式碼：presentation_delay.html](#)

```
button.addEventListener('click', () => {  
  score.incrementAndUpdateUI();  
  requestAnimationFrame(() => {  
    blockFor(1000);  
  });  
});
```

說明/活動

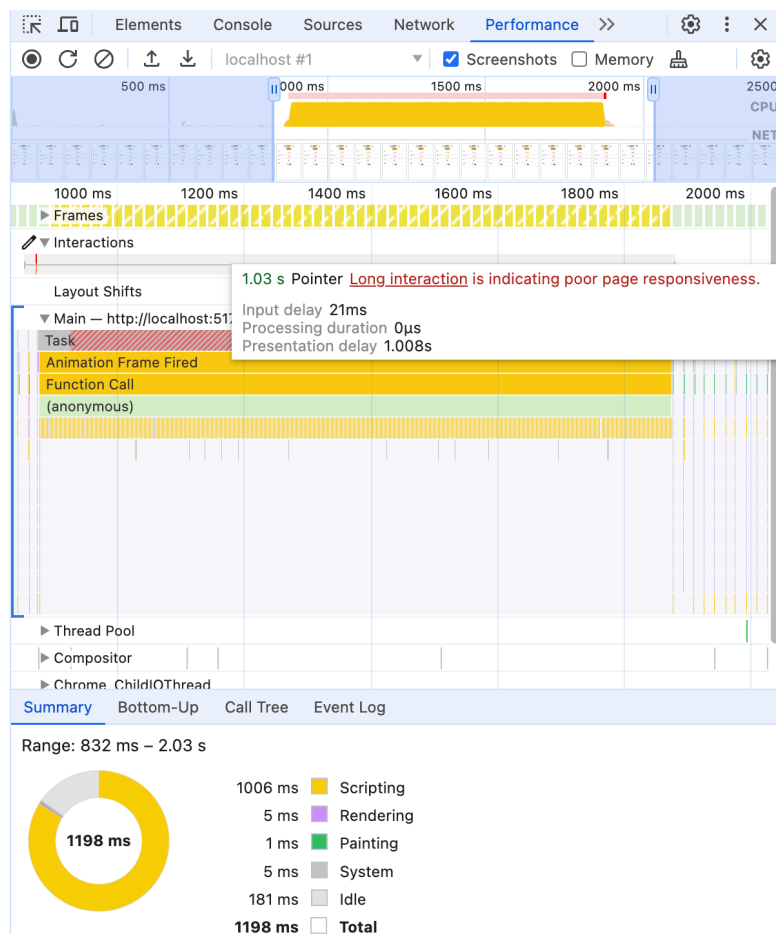
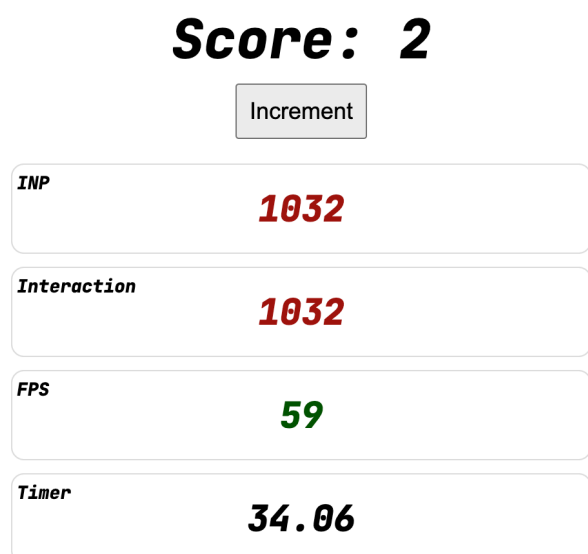
11. 簡報延遲實驗結果

查看完整程式碼：[presentation_delay.html](#)

```
button.addEventListener('click', () => {
  score.incrementAndUpdateUI();
  requestAnimationFrame(() => {
    blockFor(1000);
  });
});
```

互動時間仍為一秒，這是怎麼回事？

`requestAnimationFrame` 會在下次繪製前要求回呼。由於 INP 測量的是從互動到下一個顯示的內容之間的時間，因此 `blockFor(1000)` 中的 `requestAnimationFrame` 會持續封鎖下一個顯示的內容，時間長達一秒。



不過，請注意以下兩點：

- 將滑鼠懸停在圖表上，您會發現所有互動時間現在都花在「呈現延遲」上，因為主執行緒封鎖發生在事件監聽器傳回後。
- 主執行緒活動的根源不再是點擊事件，而是「Animation Frame Fired」。

12. 診斷互動

在這個測試頁面上，回應速度非常明顯，有分數、計時器和計數器 UI，但測試一般頁面時，回應速度就比較不明顯。

如果互動時間過長，我們不一定能找出原因。可能的原因包括：

- 輸入延遲？
- 事件處理時間？
- 簡報顯示延遲？

在任何網頁上，您都可以使用開發人員工具評估網頁的回應性。如要養成習慣，請嘗試以下流程：

1. 照常瀏覽網頁。
2. 請留意 DevTools 效能面板[即時指標檢視畫面](#)中的「互動」記錄。
3. 如果發現互動效果不佳，請嘗試重複執行：
 - 如果無法重複執行，請使用互動記錄取得洞察資料。
 - 如果可以重現，請在「效能」面板中記錄追蹤記錄。

所有延誤

請試著在頁面中加入一些上述問題：

[查看完整程式碼：all_the_things.html](#)

```
setInterval(() => {
  blockFor(1000);
}, 3000);

button.addEventListener('click', () => {
  blockFor(1000);
  score.incrementAndUpdateUI();

  requestAnimationFrame(() => {
    blockFor(1000);
  });
});
```

然後使用控制台和效能面板診斷問題！

13. 實驗：非同步工作

由於您可以在互動中啟動非視覺效果 (例如發出網路要求、啟動計時器或只是更新全域狀態)，當這些效果最終更新網頁時，會發生什麼情況？

只要允許在互動後進行下一次繪製，即使瀏覽器決定實際上不需要新的算繪更新，互動指標也會停止測量。

如要試用這項功能，請繼續從點擊事件監聽器更新 UI，但從逾時時間執行封鎖工作。

[查看完整程式碼：timeout_100.html](#)

```
button.addEventListener('click', () => {  
  score.incrementAndUpdateUI();  
  
  setTimeout(() => {  
    blockFor(1000);  
  }, 100);  
});
```

接下來呢？

14. 非同步工作實驗結果

查看完整程式碼：[timeout_100.html](#)

```
button.addEventListener('click', () => {
  score.incrementAndUpdateUI();

  setTimeout(() => {
    blockFor(1000);
  }, 100);
});
```

Score: 1

Increment

INP

24

Interaction

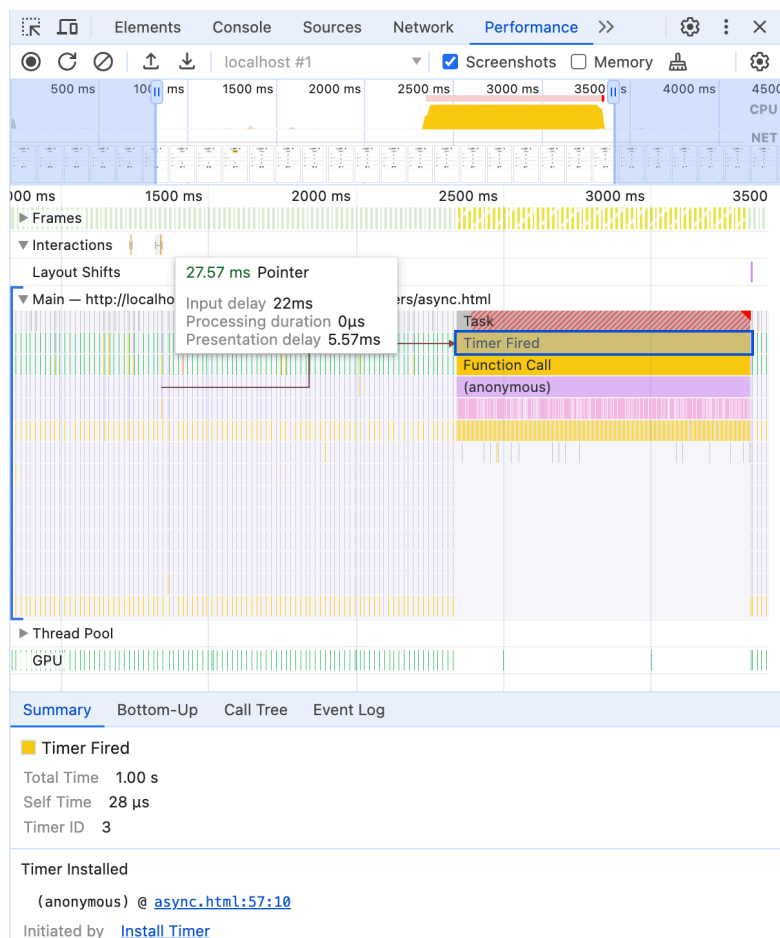
24

FPS

60

Timer

27.45



由於 UI 更新後主執行緒會立即可用，因此互動時間現在很短。長時間封鎖工作仍會執行，只是會在繪製後執行，因此使用者會立即收到 UI 回饋。

經驗：如果無法移除，至少要移動！

方法

我們是否可以做得比固定 100 毫秒的 `setTimeout` 更出色？我們可能還是希望程式碼盡快執行，否則就應該直接移除程式碼！

目標：

- 互動將執行 `incrementAndUpdateUI()`。
- `blockFor()` 會盡快執行，但不會阻礙下一次繪製。

- 這樣一來，行為就會可預測，不會發生「神奇逾時」的情況。

您可以透過下列方式達成此目標：

- `setTimeout(0)`
- `Promise.then()`
- `requestAnimationFrame`
- `requestIdleCallback`
- `scheduler.postTask()`

「requestPostAnimationFrame」

與單獨使用 `requestAnimationFrame` 不同 (這會嘗試在下次繪製前執行，通常仍會導致互動緩慢)，`requestAnimationFrame` + `setTimeout` 可為 `requestPostAnimationFrame` 提供簡單的 polyfill，在下次繪製後執行回呼。

[查看完整程式碼：raf+task.html](#)

```
function afterNextPaint(callback) {
  requestAnimationFrame(() => {
    setTimeout(callback, 0);
  });
}

button.addEventListener('click', () => {
  score.incrementAndUpdateUI();

  afterNextPaint(() => {
    blockFor(1000);
  });
});
```

為了符合人體工學，您甚至可以將其包裝在 Promise 中：

[查看完整程式碼：raf+task2.html](#)

```
async function nextPaint() {
  return new Promise(resolve => afterNextPaint(resolve));
}

button.addEventListener('click', async () => {
  score.incrementAndUpdateUI();

  await nextPaint();
  blockFor(1000);
});
```


15. 多次互動 (和暴怒點擊)

雖然將長時間阻斷工作移到其他地方有助於解決問題，但這些工作仍會阻斷網頁，影響日後的互動，以及許多其他網頁動畫和更新。

再次嘗試使用網頁的非同步封鎖工作版本 (或您自己的版本，如果您在上一個步驟中想出自己的工作延遲變化版本)：

[查看完整程式碼：timeout_100.html](#)

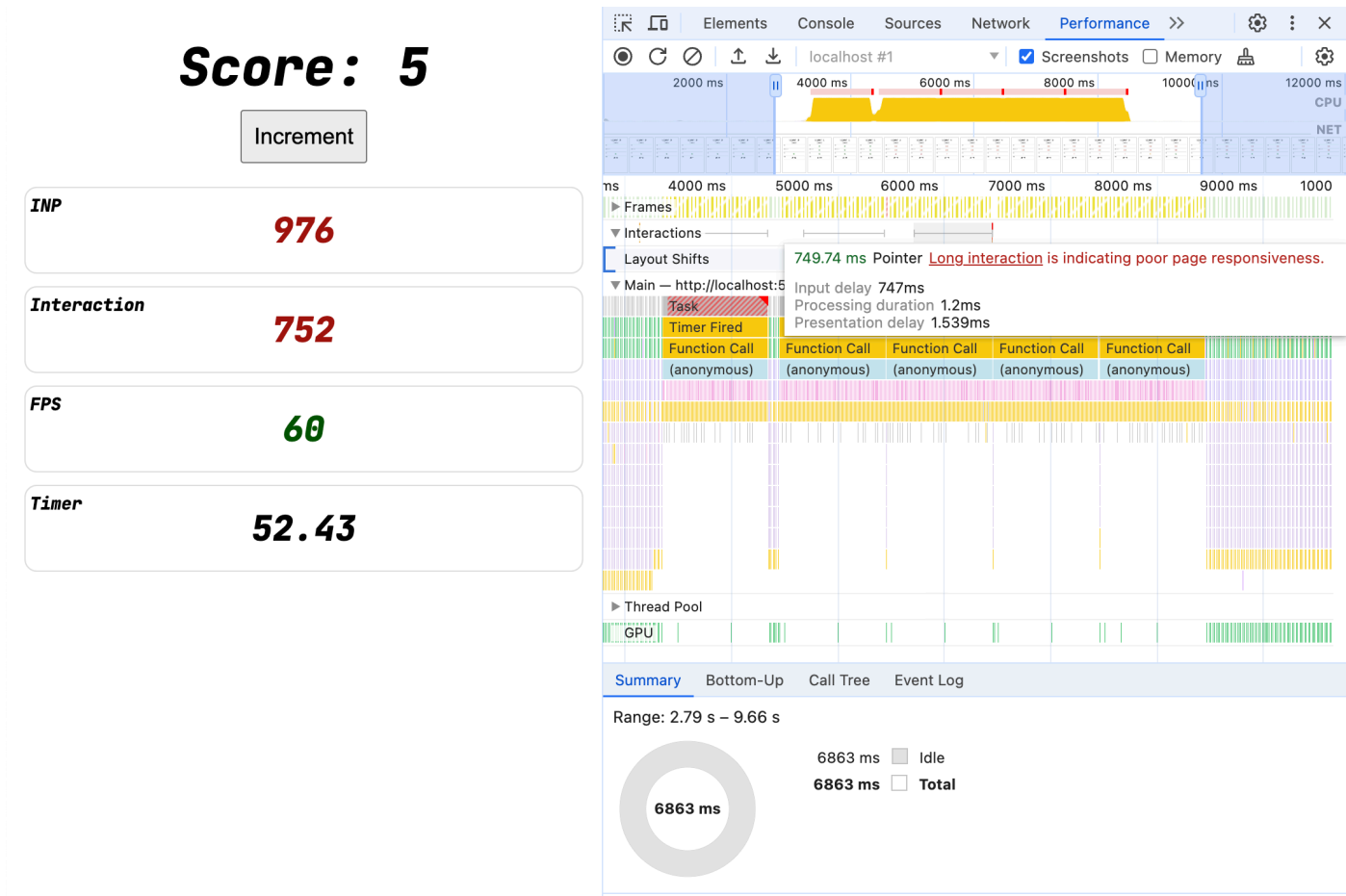
```
button.addEventListener('click', () => {
  score.incrementAndUpdateUI();

  setTimeout(() => {
    blockFor(1000);
  }, 100);
});
```

如果快速點選多次會發生什麼情況？

效能追蹤

每次點擊都會將一秒長的作業加入佇列，確保主執行緒遭到封鎖一段時間。



如果這些耗時的工作與新點擊重疊，即使事件監聽器本身幾乎會立即傳回，互動速度仍會變慢。我們已建立與先前輸入延遲實驗相同的情況。但這次的輸入延遲並非來自 `setInterval`，而是由先前的事件監聽器觸發的工作所致。

視使用者多次點按的原因而定，有時也稱為「暴怒點按」。資料顯示，[網頁回應速度緩慢會增加激怒點擊次數](#)，而激怒點擊次數越多，回應速度就越慢！

策略

理想情況下，我們希望完全移除長時間工作！

- 完全移除不必要的程式碼，尤其是指令碼。
 - 最佳化程式碼，避免執行耗時的工作。
 - 在收到新的互動時中止過時的工作。
-

16. 策略 1：去抖動

這是經典策略。如果互動接踵而來，且處理或網路效應成本高昂，請刻意延遲啟動作業，以便取消並重新啟動。這種模式適用於自動完成欄位等使用者介面。

- 使用 `setTimeout` 延遲啟動耗用資源的工作，並設定計時器 (例如 500 到 1000 毫秒)。
- 請儲存計時器 ID。
- 如果收到新的互動，請使用 `clearTimeout` 取消先前的計時器。

查看完整程式碼：[debounce.html](#)

```
let timer;
button.addEventListener('click', () => {
  score.incrementAndUpdateUI();

  if (timer) {
    clearTimeout(timer);
  }
  timer = setTimeout(() => {
    blockFor(1000);
  }, 1000);
});
```

效能追蹤

Score: 5

Increment

INP

24

Interaction

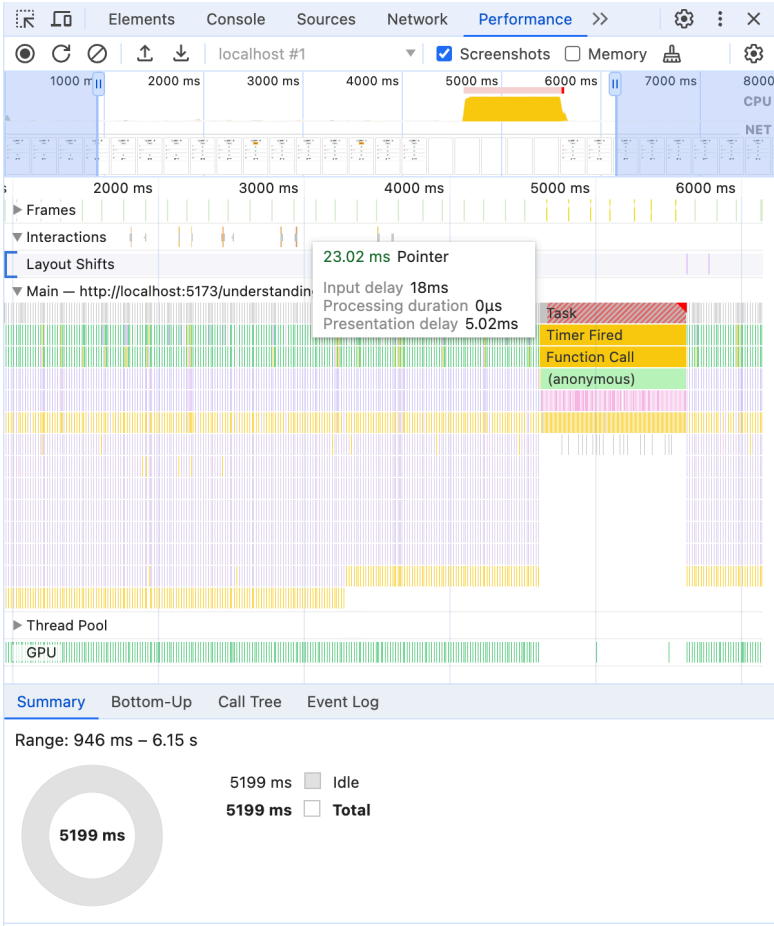
24

FPS

60

Timer

36.28



即使多次點選，也只會執行一項 `blockFor` 工作，且必須等待一整秒沒有任何點選動作，才會執行工作。如果互動會突然湧入，例如在文字輸入欄中輸入內容，或是預期項目目標會快速獲得多次點擊，建議預設使用這項策略。

17. 策略 2：中斷長時間執行的工作

但如果點擊發生在去抖動時間過後，且正好落在長時間任務的中間，就可能因為輸入延遲而導致互動速度緩慢。

理想情況下，如果互動發生在工作期間，我們會暫停忙碌的工作，以便立即處理任何新的互動。我們該怎麼做？

有些 API (例如 `isInputPending`) 也能達到類似效果，但一般來說，最好將長時間執行的工作分割成多個區塊。

大量 `setTimeout`

第一次嘗試：做簡單的事。

[查看完整程式碼：small_tasks.html](#)

```
button.addEventListener('click', () => {
  score.incrementAndUpdateUI();

  requestAnimationFrame(() => {
    setTimeout(() => blockFor(100), 0);
    setTimeout(() => blockFor(100), 0);
    setTimeout(() => blockFor(100), 0);
    setTimeout(() => blockFor(100), 0);
    setTimeout(() => blockFor(100), 0);
    setTimeout(() => blockFor(100), 0);
    setTimeout(() => blockFor(100), 0);
    setTimeout(() => blockFor(100), 0);
    setTimeout(() => blockFor(100), 0);
    setTimeout(() => blockFor(100), 0);
    setTimeout(() => blockFor(100), 0);
  });
});
```

這項功能可讓瀏覽器個別排定每項工作的時間，並優先處理輸入內容！

Score: 5

Increment

INP

104

Interaction

96

FPS

59

Timer

38.18



我們又回到五次點擊需要五秒鐘的完整工作，但每次點擊的一秒鐘工作已分成十個 100 毫秒的工作。因此，即使多個互動與這些工作重疊，任何互動的輸入延遲都不會超過 100 毫秒！瀏覽器會優先處理傳入的事件監聽器，而非 `setTimeout` 工作，因此互動仍能保持回應。

如果您要排定個別進入點 (例如在應用程式載入時間呼叫大量獨立功能)，這個策略就特別實用。預設情況下，只要載入指令碼並在指令碼評估時間執行所有項目，所有項目就會在巨大的長時間任務中執行。

不過，如果程式碼緊密耦合 (例如使用共用狀態的 `for` 迴圈)，這種策略就無法有效拆解。

現在支援 `yield()`

不過，我們可以運用新版 `async` 和 `await`，輕鬆將「產生點」新增至任何 JavaScript 函式。

例如：

[查看完整程式碼：yieldy.html](#)

```
// Polyfill for scheduler.yield()
async function schedulerDotYield() {
  return new Promise(resolve => {
    setTimeout(resolve, 0);
  });
}

async function blockInPiecesYieldy(ms) {
  const ms_per_part = 10;
  const parts = ms / ms_per_part;
  for (let i = 0; i < parts; i++) {
    await schedulerDotYield();

    blockFor(ms_per_part);
  }
}

button.addEventListener('click', async () => {
  score.incrementAndUpdateUI();
  await blockInPiecesYieldy(1000);
});
```

與先前一樣，主執行緒會在處理完一組工作後產生，瀏覽器也能回應任何傳入的互動，但現在只需要 `await schedulerDotYield()`，不必使用個別的 `setTimeout`，因此即使在 `for` 迴圈中也能輕鬆使用。

現在支援 `AbortController()`

這樣做可以運作，但即使有新的互動傳入，且可能改變需要完成的工作，每次互動仍會排定更多工作。

採用去抖動策略後，我們會在每次有新互動時取消先前的逾時。Can we do something similar here? 其中一種做法是使用 `AbortController()`：

[查看完整程式碼：aborty.html](#)

```

// Polyfill for scheduler.yield()
async function schedulerDotYield() {
  return new Promise(resolve => {
    setTimeout(resolve, 0);
  });
}

async function blockInPiecesYieldyAborty(ms, signal) {
  const parts = ms / 10;
  for (let i = 0; i < parts; i++) {
    // If AbortController has been asked to stop, abandon the current loop.
    if (signal.aborted) return;

    await schedulerDotYield();

    blockFor(10);
  }
}

let abortController = new AbortController();

button.addEventListener('click', async () => {
  score.incrementAndUpdateUI();

  abortController.abort();
  abortController = new AbortController();

  await blockInPiecesYieldyAborty(1000, abortController.signal);
});

```

當點擊事件傳入時，系統會啟動 `blockInPiecesYieldyAborty` `for` 迴圈，執行需要完成的任何工作，同時定期產生主執行緒，讓瀏覽器持續回應新的互動。

當第二次點擊發生時，第一個迴圈會標示為已取消 (使用 `AbortController`)，並啟動新的 `blockInPiecesYieldyAborty` 迴圈。下次排定再次執行第一個迴圈時，系統會發現 `signal.aborted` 現在為 `true`，並立即傳回，不會執行後續工作。

Score: 5

Increment

INP

32

Interaction

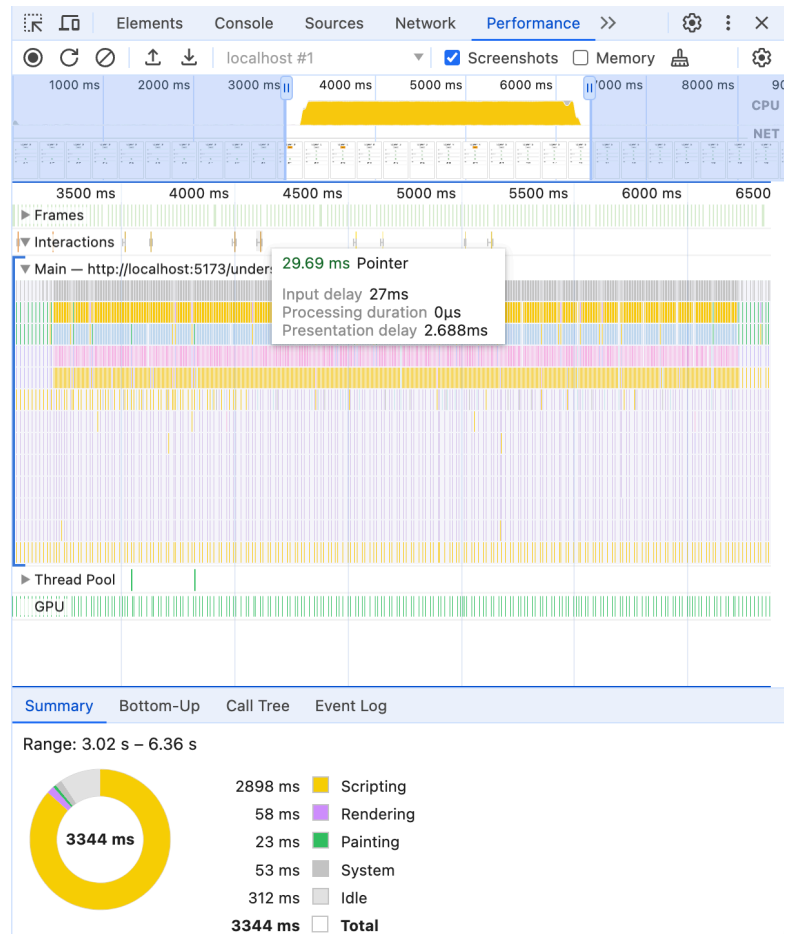
32

FPS

59

Timer

51.34



注意：產生結果有助於提升回應速度，因為這樣可以排定初始意見回饋並允許算繪，但如果產生過多結果，可能會影響整體 CPU 輸送量。您不會想在每一行程式碼之間產生收益！

18. 結論

將所有長時間執行的工作拆開，網站就能回應新的互動。這樣您就能快速提供初步意見，並決定是否要中止進行中的工作。有時這表示要將進入點排定為個別工作。有時這表示要在方便的位置新增「產生」點。

記住

- INP 會評估所有互動。
- 系統會測量從輸入到下一個顯示的內容之間的每次互動，也就是使用者看到的回應。
- 輸入延遲、事件處理時間和呈現延遲都會影響互動回應速度。
- 您可以使用開發人員工具輕鬆評估 INP 和互動細目！

策略

- 網頁上沒有長時間執行的程式碼 (長時間執行的工作)。
- 將不必要的程式碼移出事件監聽器，直到下一次繪製完成為止。
- 確保瀏覽器能有效率地更新算繪內容。

瞭解詳情

- [web.dev：Interaction to Next Paint](#)
 - [web.dev：最佳化 Interaction to Next Paint](#)
 - [本程式碼研究室的原始版本](#)
-